

Neo-Bank Backend Simulator — Final V1 Project Report

Author: Dev | **Scope:** End-to-End Fintech Backend System Design

1. Executive Summary

This project implements a production-style backend for a digital neo-bank using simulated money but real-world financial engineering principles: double-entry ledger accounting, idempotent APIs, webhook-driven payments, reconciliation pipelines, fraud rules, admin tooling, chatbot interfaces, strict security hardening, and AWS deployment readiness. The backend is built as a Spring Boot modular monolith to preserve transactional integrity while remaining evolvable into microservices.

2. Design Goals

Functional: wallet management, transfers, payment gateway sandbox, subscriptions, disputes, fraud detection, chatbot support, admin tooling.

Non-functional: correctness-first, idempotency, observability, auditability, security, deterministic failure recovery, scalability.

3. Architecture Overview

Clients interact with a REST API layer that enforces RBAC and limits. Domain services execute business logic inside DB transactions, persist ledger entries, and emit outbox events. Async workers consume events for reconciliation, notifications, and analytics.

4. Internal Modules

- 1 auth
- 2 users
- 3 wallets
- 4 ledger
- 5 transfers
- 6 payments
- 7 subscriptions
- 8 disputes
- 9 fraud
- 10 reconciliation
- 11 chatbot
- 12 admin
- 13 audit
- 14 notifications
- 15 ratelimit
- 16 security
- 17 infra

5. Infrastructure Stack

Spring Boot 3, PostgreSQL, Redis, Kafka/RabbitMQ/SQS, Quartz scheduler, Docker, AWS ECS/EC2, RDS, S3, CloudWatch, Secrets Manager, GitHub Actions.

6. Ledger-First Financial Model

Balances are derived from immutable ledger rows. All money movements create paired debit and credit entries; corrections are handled by reversal rows.

7. Subscription Model

Tier	Daily Limit	Benefits
Free	■10k/day	basic transfers
Premium	■2L/day	analytics, APIs, priority queues

8. Database Schema

users: id, email, password_hash, role, created_at

wallets: id, user_id, currency, status, snapshot_balance, daily_limit

ledger_entries: id, wallet_id, amount, entry_type, transaction_group_id, reference_type, reference_id, created_at

payments: id, gateway_order_id, gateway_payment_id, status, amount, idempotency_key

subscriptions: id, user_id, plan, status, started_at, expires_at

disputes: id, transaction_group_id, status, reason, opened_at, resolved_at

audit_logs: id, actor_id, action, entity_type, entity_id, timestamp

idempotency_keys: key, request_hash, response, expires_at

reconciliation_reports: run_date, mismatch_count, status

chatbot_logs: user_id, intent, action, timestamp

outbox_events: id, type, payload, status

9. Transfer Flow

Authenticate → validate + idempotency → enforce plan/limits → lock wallets → insert debit/credit ledger rows → commit → audit log → outbox event → async workers.

10. Payment/Webhook Flow

Create order → user pays → webhook received → signature verification → deduplication → ledger credit → wallet update → event emission.

11. Idempotency Strategy

State-changing APIs require idempotency keys stored with request hashes and cached responses; duplicate calls return the original result.

12. Async Processing

Outbox pattern and queue workers power notifications, analytics, fraud scoring, reconciliation, PDF generation, and chatbot follow-ups with retries and DLQs.

13. Reconciliation Engine

Nightly recomputation of balances versus wallet snapshots, mismatch detection, wallet freezing, admin alerts, and report generation.

14. Ticket/Dispute System

Workflow: OPEN → UNDER_REVIEW → RESOLVED/REJECTED. Opening a dispute freezes funds; resolution posts reversal ledger entries and updates status.

15. Fraud & Risk Rules

Velocity checks, abnormal amounts, repeated failures, rapid drains, and geographic anomalies feed admin queues.

16. Chatbot Engine

Policy-guarded intent engine routes user requests to domain services with confirmations, RBAC, and audit logging.

17. Security Hardening

JWT auth, RBAC, Redis rate limits, DTO validation, encrypted secrets, webhook HMAC verification, OWASP Top-10 defenses, CSRF protection, and audit trails.

18. Observability

Structured logs, metrics dashboards, tracing, queue-depth alerts, and reconciliation alarms.

19. Deployment Strategy

Local Docker Compose; production ECS/EC2, RDS PostgreSQL, Redis, S3, CI/CD, blue-green deploys.

20. Scaling & Microservice Evolution

Sharding wallets, partitioning ledgers, read replicas, cache layers, isolated fraud workers, and future domain-based service extraction with saga workflows.

21. Execution Roadmap

Phase	Focus
1	Ledger + transfers
2	Payments + subscriptions
3	Disputes + reconciliation
4	Fraud + chatbot
5	Infra + docs

22. Resume Description

Designed a neo-bank backend simulator with ledger-first accounting, sandbox payment gateway integration, Free/Premium tiers, dispute workflows, reconciliation engines, chatbot interfaces, OWASP-aligned security, and AWS-ready infrastructure using Spring Boot.